DESARROLLO WEB

Despliegue de backend con Django

Python, PostgreSQL y Render

Junior Perez Davila - 407721

Claudia Libertad Quispe Terrones - 407719

Samuel Andres Zona Malaver - 385387

Proyecto: mi_proyecto_routing

Repositorio: https://github.com/JuniorPD04/practica_django

Servicio: https://routing-api-kde6.onrender.com

Verificación: https://routing-api-kde6.onrender.com/authors

13 de noviembre de 2026

I. CUESTIONARIO

1. ¿Qué significa desplegar una aplicación web?

Para nosotros, desplegar una aplicación es hacer que deje de funcionar solo en nuestro computador y pueda usarse desde internet. En este proyecto tuvimos que subir el código a GitHub, configurar Render, instalar las dependencias y conectar PostgreSQL. También fue necesario ejecutar las migraciones y comprobar la dirección pública. Una forma sencilla de verificar el resultado fue abrir /authors desde el navegador y revisar que aparecieran nuestros nombres y códigos.

2. ¿Qué es Render? Ventajas y limitaciones

Render es el servicio donde publicamos nuestro backend. Lo usamos porque permite conectar el repositorio de GitHub y ofrece opciones gratuitas para una práctica como esta. Dos ventajas son que puede desplegar los cambios del repositorio y que configura HTTPS automáticamente. Sin embargo, el servicio gratuito se duerme después de 15 minutos sin uso y su base PostgreSQL gratuita expira a los 30 días. Por eso resulta útil para el taller, aunque debemos tener presentes esos límites.

3. Diferencias entre desarrollo y producción

En nuestro computador usamos runserver para desarrollar; en Render usamos Gunicorn para atender las solicitudes. Otra diferencia es la base de datos: localmente trabajamos con SQLite y en el despliegue configuramos PostgreSQL. También cambia DEBUG, que nos ayuda a encontrar errores en desarrollo, pero debe estar desactivado al publicar. Además, en producción usamos una dirección pública con HTTPS, restringimos los hosts permitidos y guardamos las claves en variables de entorno. Los archivos estáticos también necesitan una configuración que en local suele pasar desapercibida.

4. Variables de entorno y credenciales

Las variables de entorno nos permiten cambiar datos de configuración sin editar el código. Por ejemplo, guardamos allí SECRET_KEY y DATABASE_URL. Si escribiéramos una contraseña directamente en settings.py, podríamos terminar publicándola en GitHub y exponiendo el acceso a nuestra base. En local utilizamos un archivo .env que Git ignora. En Render se agregan desde Environment; nuestro Blueprint también permite generar la clave y enlazar la conexión a PostgreSQL sin escribir las credenciales en el repositorio.

5. Pasos para cambiar de motor y relación con el ORM

Primero agregamos psycopg2-binary para conectar Python con PostgreSQL y dj-database-url para leer la dirección de conexión. Luego ajustamos DATABASES, creamos la base en Render y ejecutamos las migraciones. Fueron pocos cambios porque ya trabajábamos con el ORM de Django: nuestras consultas se hacen con modelos de Python y Django las traduce al SQL del motor elegido. Eso sí, cambiar la conexión no copia los datos de SQLite. Para esta práctica dejamos una carga de datos de ejemplo en la base nueva.

6. ¿Qué es PaaS? Comparación

Un servicio PaaS nos permite publicar una aplicación sin tener que instalar y administrar todo el servidor por nuestra cuenta. Nosotros entregamos el código y la configuración, mientras la plataforma se ocupa de buena parte de la infraestructura. Render nos facilitó alojar Django y PostgreSQL dentro del mismo servicio de plataforma. Vercel también permite publicar aplicaciones y soporta Django, aunque suele utilizarse mucho para frontends y funciones. Antes de elegir entre ambas conviene revisar cómo ejecutan el backend y qué límites tiene cada plan.

7. ¿Por qué se obtiene HTTPS automáticamente?

La API tiene HTTPS porque Render se encarga de generar y administrar el certificado del dominio. Cuando una persona entra a nuestra dirección, la conexión segura llega primero a la infraestructura de Render y luego la solicitud se envía a Django. Por eso no tuvimos que comprar ni instalar un certificado manualmente. En settings.py indicamos cómo reconocer el protocolo que informa el proxy, para que Django sepa que la solicitud original llegó por HTTPS.

8. Cold start

El cold start es la espera que ocurre cuando alguien vuelve a usar el servicio después de que Render lo haya suspendido por inactividad. En el plan gratuito esto puede pasar tras 15 minutos sin solicitudes, y reactivarlo puede tomar alrededor de un minuto. Para una exposición, conviene abrir /authors un poco antes y esperar a que responda. Para un usuario significa que la primera visita puede ser lenta, aunque las siguientes solicitudes ya se atiendan con el servicio activo.

9. Alternativas para conservar la base sin costo

Dos alternativas que podemos considerar son Neon y Supabase. Ambas ofrecen planes gratuitos de PostgreSQL con sus propios límites. Neon permite mantener una base sin el plazo fijo de 30 días de Render, siempre que se respeten sus cuotas. Supabase también puede servir para continuar el proyecto, aunque sus proyectos gratuitos pueden pausarse por inactividad. Para cambiarnos tendríamos que crear la nueva base, transferir los datos antes de que expire la anterior y actualizar DATABASE_URL. También habría que ajustar el Blueprint para que conserve esa nueva conexión.

10. Seguridad básica al publicar la API

Al publicar la API debemos desactivar DEBUG, mantener privadas las claves y restringir los dominios permitidos. También necesitamos HTTPS, validar los datos que recibimos y comprobar los permisos de cada ruta. No basta con que exista un login: cada operación que maneje información privada debe exigir la autorización correspondiente. En nuestro proyecto hay rutas antiguas del juego que siguen siendo públicas y deben revisarse si se amplía su uso. La ruta /authors sí debe ser pública, porque así lo pide la actividad.

11. ¿Por qué Gunicorn en lugar de runserver?

Usamos runserver mientras desarrollamos porque es cómodo para probar cambios y encontrar errores. Para publicar elegimos Gunicorn, que está preparado para ejecutar la aplicación mediante procesos de trabajo y atender solicitudes en un entorno de producción. En Render arranca nuestro módulo WSGI y escucha en el puerto que asigna la plataforma. Así podemos seguir usando runserver en nuestro computador y dejar el servidor de producción configurado en el despliegue.

12. Función de render.yaml

El archivo render.yaml reúne las instrucciones para crear el servicio web y su base de datos. Allí indicamos la carpeta del proyecto de routing, los planes, los comandos de instalación y arranque, y las variables necesarias. Nos resultó útil porque el repositorio tiene dos proyectos y debíamos señalar cuál desplegar. Tener esta configuración en Git también permite revisarla y repetir el despliegue con menos pasos manuales. Las contraseñas reales se siguen configurando de forma privada.

13. Cambios concretos en settings.py

En settings.py hicimos que SECRET_KEY y DEBUG se lean desde el entorno. También configuramos ALLOWED_HOSTS para aceptar la dirección asignada por Render. En DATABASES dejamos PostgreSQL cuando existe DATABASE_URL y SQLite para el desarrollo local. Agregamos STATIC_ROOT como destino de collectstatic y configuramos WhiteNoise en STORAGES y MIDDLEWARE para servir los archivos estáticos. Por último, ajustamos el reconocimiento de HTTPS detrás del proxy y las cookies seguras. Cada cambio permite que el mismo proyecto funcione en local y al publicarse.

14. Propósito de WhiteNoise

WhiteNoise permite que el backend entregue archivos como CSS y JavaScript cuando está publicado. En desarrollo Django nos facilita esa tarea, pero al desactivar DEBUG necesitamos prepararla para producción. Primero collectstatic reúne los archivos en STATIC_ROOT y luego WhiteNoise los sirve. Esto ayuda, por ejemplo, a que el administrador cargue con sus estilos. También comprime los recursos y facilita su caché. Los archivos que suban los usuarios necesitarían una solución de almacenamiento persistente aparte.

15. Modelos y migraciones al cambiar a PostgreSQL

No tuvimos que cambiar los modelos ni reescribir las migraciones porque ya describían las tablas y relaciones mediante las herramientas de Django. El ORM se encarga de traducir nuestras operaciones al motor elegido, y las migraciones pueden crear esa estructura en PostgreSQL. En esta práctica lo comprobamos también con una base PostgreSQL temporal. Esto no significa que cualquier consulta sea portable: si usáramos SQL exclusivo de SQLite, habría que revisarlo. Además, crear las tablas y trasladar sus datos son tareas distintas.

II. EVIDENCIA DEL DESPLIEGUE

La siguiente captura muestra la consulta a /authors desde el navegador. La respuesta presenta los nombres y códigos de los tres integrantes, sin que sea necesario iniciar sesión ni enviar un token de autenticación.

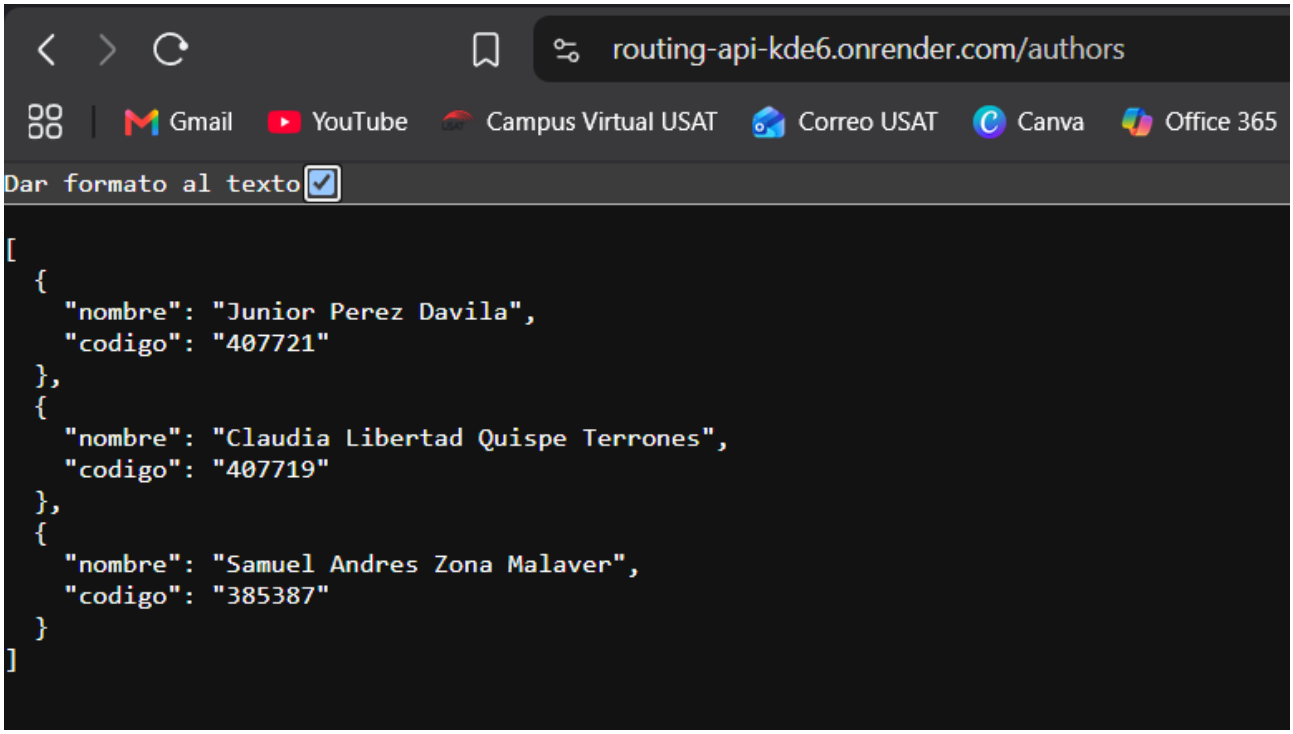


Fig. 1. Respuesta pública de /authors sin credenciales.

Dirección consultada: https://routing-api-kde6.onrender.com/authors.